MODAL GUITAR LAB

Construire une exploration musicale interactive

CHAPITRE 02
Visualisation des gammes sur le manche

Des intervalles musicaux aux classes CSS dynamiques
React · TypeScript strict · MIDI · CSS

Séance réalisée le 23 octobre 2026

Documentation rédigée le 24 octobre 2026 · Statut : séance validée, build réussi

Projet personnel, sans backend ni audio à ce stade.
Chapitre conçu pour être ajouté au carnet cumulatif du projet.

Mode d’emploi de ce chapitre

Ce document conserve à la fois **ce qui a été fait** pendant la séance, **la raison de chaque choix** et **les étapes nécessaires pour le reconstruire**. Il fait suite au chapitre 01 (manche interactif et calcul des notes MIDI).

Le code reproduit dans le corps du chapitre s’appuie sur les extraits effectivement fournis pendant l’échange. L’annexe propose, elle, une **implémentation pédagogique autonome** pour repartir d’un projet Vite vierge : elle est volontairement explicitée comme un exemple de reconstruction, et non comme une copie exacte de tous les fichiers du dépôt.

> **Périmètre vérifié.** Affichage de Mi mineur naturel sur six cordes et cases 0 à 12 ; toniques bleues, notes de gamme vertes, sélection orange ; vérifications manuelles réussies et `pnpm build` validé. Aucun test automatisé n’a été indiqué dans cette séance.

Sommaire

01 Point de départ et objectif
02 Modèle musical : intervalles plutôt que notes
03 Classes de hauteur et piège du modulo
04 Construction de scales.ts
05 Connexion à la note sélectionnée
06 Affichage de la gamme sur tout le manche
07 Mise en évidence des toniques
08 Vérifications et difficultés
09 Journal des décisions et limites
10 Procédure de reconstruction
A Annexe : base autonome reproductible
11 Passage à la séance 3

Convention de lecture. Les chemins, signatures et extraits de code issus de la séance sont présentés tels qu’ils ont été établis, avec seulement des corrections typographiques mineures. Les fichiers non fournis intégralement sont signalés.

01 Point de départ et objectif

État hérité de la séance 1

Application locale React + TypeScript strict, initialisée avec Vite et pnpm. Le manche présente six cordes, les cases 0 à 12 et un état de sélection géré par `useState`. À partir de la corde et de la case, le programme obtient la note, l’octave et la valeur MIDI.

Élément	État au début de la séance 2
Modèle corde	`GuitarString` : position 1 à 6 et `openMidi`
Accordage	E4 / B3 / G3 / D3 / A2 / E2 (cordes 1 → 6)
Logique existante	`getMidiNote`, `getNoteName`, `getOctave`, `getFullNoteName`, `checkMidiRange`

Élément	État au début de la séance 2
Vue	`GuitarNeck.tsx` : `guitarStrings.map` puis `fretNumbers.map`
Contraintes TypeScript	`strict` et `noUncheckedIndexedAccess` activés

Le nouveau besoin est orthogonal à la sélection : **chaque position doit savoir si elle appartient à une gamme**, même lorsqu'elle n'est pas sélectionnée. Pour commencer, la gamme et sa tonique restent fixes : Mi mineur naturel (E éolien).

Objectif et non-objectifs

- Obtenir une représentation musicale exploitable, puis reconnaître une note MIDI dans une gamme, quelle que soit son octave.
- Afficher toutes les positions appartenant à la gamme et distinguer la tonique, sans casser la sélection existante.
- Ne pas introduire d'audio, de tablature, de composition, de sélecteur de gamme ou de backend dans ce chapitre.

02 Modèle musical : intervalles plutôt que notes

Une première piste consistait à stocker directement les classes de hauteur des sept notes de Mi mineur. La difficulté rencontrée était d'en comprendre l'intérêt. Le choix retenu pendant la séance est plus musical et plus extensible : **décrire une gamme à partir de ses intervalles**.

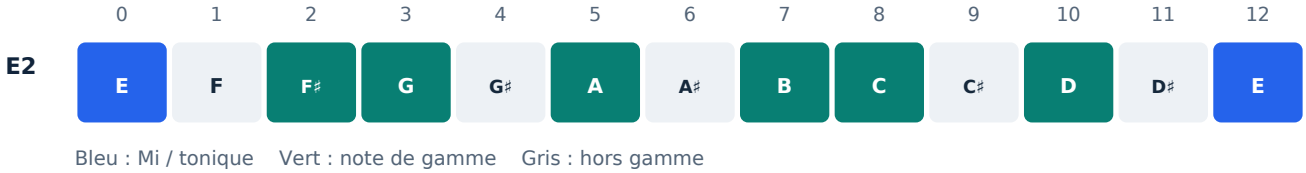
Deux représentations qu'il ne faut pas confondre

Vue	Valeurs	Sens
Succession de tons et demi-tons	`[2, 1, 2, 2, 1, 2, 2]`	Distance entre deux notes consécutives. Le dernier 2 conduit à l'octave.
Offsets depuis la tonique	`[0, 2, 3, 5, 7, 8, 10]`	Distance de chacune des sept notes à la tonique, en demi-tons.

La seconde représentation a été choisie car l'algorithme demande : « la distance entre cette note et ma tonique figure-t-elle parmi les intervalles autorisés ? ». Cela rend l'appartenance facile à tester avec `includes()`.

Note	E	F♯	G	A	B	C	D	E (octave)
Distance en demi-tons	0	2	3	5	7	8	10	12
Degré	1	2	♭3	4	5	♭6	♭7	1

Une octave vaut douze demi-tons, soit douze cases sur une même corde. La représentation par intervalles est donc directement lisible sur la guitare : corde 6 à vide = E2, cases 2 = F♯2, 3 = G2, 5 = A2, 7 = B2, 8 = C3, 10 = D3 et 12 = E3.



Bleu : Mi / tonique Vert : note de gamme Gris : hors gamme

Pourquoi ce choix prépare la suite. Le modèle ne contient aucune note absolue. La même série d'intervalles pourra être appliquée à La, Ré ou Fa♯. Pour changer de mode, il suffira plus tard de remplacer cette série par une autre ; cette généralisation n'est pas encore implémentée.

03 Classes de hauteur et piège du modulo

Les valeurs MIDI encodent aussi l'octave : E2 vaut 40, E3 vaut 52 et E4 vaut 64. Pourtant ce sont toutes des toniques de Mi mineur. Un contrôle direct contre des valeurs MIDI fixes ne suffirait pas.

Ramener une distance à l'intérieur d'une octave

Le calcul `midiNote - tonicMidi` produit une distance orientée en demi-tons. Pour reconnaître les notes équivalentes à une octave près, on prend cette distance modulo 12.

Comparaison à E4 (64)	Différence	Intervalle ramené sur 12
E5 (76)	12	0 : tonique
G5 (79)	15	3 : tierce mineure
G3 (55)	−9	3 : tierce mineure
D3 (50)	−14	10 : septième mineure

La difficulté rencontrée : −21 % 12

Au cours de la séance, le calcul `-21 % 12` a fait hésiter. Mathématiquement, le modulo positif vaut 3. Mais en JavaScript, `%` renvoie le **reste** de la division tronquée vers zéro : `-21 % 12` vaut **−9**.

Exemples à vérifier dans la console

```
-21 % 12            // -9  (JavaScript)
(-21 % 12 + 12) % 12 // 3   (modulo positif)
15 % 12             // 3
-14 % 12            // -2
(-14 % 12 + 12) % 12 // 10
```

L'expression `((value % mod) + mod) % mod` normalise un reste négatif lorsque `mod` est positif. Le second modulo est indispensable pour ne pas produire 12 pour un intervalle équivalent à zéro.

Point de vigilance. Ici `mod` vaut toujours 12 et les notes MIDI utilisées sont des entiers. La fonction utilitaire n'a pas été conçue ou testée comme une API mathématique générale pour un diviseur nul ou négatif.

04 La nouvelle brique : src/music/scales.ts

La logique des gammes a été placée dans un nouveau module distinct de `notes.ts`. Ce choix garde le calcul de hauteur MIDI indépendant de la notion d'appartenance à une gamme.

Code final fourni pendant la séance (présentation nettoyée)

```
// Intervalles en demi-tons depuis la tonique.
export const NATURAL_MINOR_INTERVALS = [0, 2, 3, 5, 7, 8, 10];

function modulo(value: number, mod: number) {
    return ((value % mod) + mod) % mod;
}

export function isNoteInNaturalMinor(
    midiNote: number,
    tonicMidi: number
): boolean {
    const distance = modulo(midiNote - tonicMidi, 12);
    return NATURAL_MINOR_INTERVALS.includes(distance);
}
```

Lecture ligne par ligne

- `NATURAL_MINOR_INTERVALS` : distances autorisées, exprimées depuis la tonique ; les octaves n'y figurent pas.
- `modulo()` : petit utilitaire local, non exporté car seul ce fichier en a besoin.
- `isNoteInNaturalMinor()` : fonction pure ; deux nombres en entrée, un booléen en sortie ; aucun état React.
- `includes(distance)` : vérifie l'appartenance à la liste ; ne retourne pas la distance, mais bien `true` ou `false`.

TypeScript infère `number[]` pour la constante et `number` pour le retour de `modulo`. Un `readonly` ou `as const` serait envisageable, mais non nécessaire à cet incrément.

Cas de contrôle représentatifs

Appel	Résultat	Pourquoi
`(76, 64)`	**true**	E5 : 12 → 0
`(79, 64)`	**true**	G5 : 15 → 3
`(55, 64)`	**true**	G3 : −9 → 3
`(50, 64)`	**true**	D3 : −14 → 10
`(53, 64)`	**false**	F3 : −11 → 1
`(40, 64)`	**true**	E2 : −24 → 0

05 Premier branchement React : la note sélectionnée

Avant de colorer le manche entier, la fonction musicale a été appelée depuis l'affichage de la position actuellement sélectionnée. C'était un incrément volontairement petit : vérifier la logique avec le système de sélection déjà en place.

Extrait explicatif de buildOutput (renommage de lisibilité)

```
function buildOutput(
    selectedPosition: SelectedPosition,
    selected: GuitarString
) {
    const midi = getMidiNote(selected.openMidi, selectedPosition.fret);
    const note = getFullNoteName(midi);
    const scaleStatus = isNoteInNaturalMinor(midi, 64)
        ? 'Dans la gamme'
        : 'Hors gamme';

    return <span>{`NOTE: ${note} MIDI: ${midi} ${scaleStatus}`}</span>;
}
```

Dans l'extrait réellement proposé pendant la séance, la variable s'appelait `isScale` ; le renommage en `scaleStatus` est seulement une recommandation de lisibilité, car la valeur contient du texte et non un booléen.

Concept React : valeur dérivée

L'appartenance à la gamme ne doit pas devenir un second état React. Elle se **déduit** directement de la position sélectionnée, de la hauteur MIDI et de la tonique. Ajouter `useState` pour cette valeur introduirait une synchronisation inutile. Le calcul étant très léger, aucun `useMemo` ni `useEffect` n'est justifié.

Cet incrément a été vérifié sur la corde grave : E2, F♯2, G2, G♯2 et E3. Le texte « Dans la gamme » ou « Hors gamme » correspondait à la note cliquée.

06 Afficher toute la gamme sur le manche

La deuxième intégration utilise la structure de rendu existante : une boucle sur les six cordes, puis une boucle sur les cases. Chaque cellule calcule sa hauteur MIDI et appelle `isNoteInNaturalMinor()`.

Extrait de structure JSX (présentation compacte)

```
guitarStrings.map((string) => (
  <div key={string.pos} className="string-row">
    <div className="string-name">
      {getFullNoteName(string.openMidi)}
    </div>
    {fretNumbers.map((fret) => {
      const isSelected = cellIsSelected(
        string.pos, fret, selectedPosition
      );
      const midiNote = getMidiNote(string.openMidi, fret);
      const note = isSelected ? getFullNoteName(midiNote) : '';
      const isInScale = isNoteInNaturalMinor(midiNote, 64);
```

```
      return (
        <div key={fret}
          className={`fret-cell ${isSelected ? 'is-selected' : ''}
            ${isInScale ? 'in-scale' : ''}`}
          onClick={() => handleFretClick(string.pos, fret)}
          data-note={note}>
          <div className="string-line"
            style={{ '--i': string.pos } as React.CSSProperties} />
        </div>
      );
    })}
  </div>
))
```

Les deux indicateurs sont indépendants : `isSelected` exprime une interaction de l'utilisateur ; `isInScale` exprime une propriété musicale. Une case hors gamme doit rester sélectionnable.

Sélectionnée ?	Dans la gamme ?	Rendu attendu
Non	Non	Case neutre
Non	Oui	Fond vert
Oui	Non	Case neutre + pastille orange
Oui	Oui	Fond vert + pastille orange

Couches CSS : cellule vs pseudo-élément

Règles essentielles : couleur de gamme + pastille de sélection

```
.in-scale {
    background-color: rgba(3, 76, 52, 0.524);
}

.is-selected::after {
    content: attr(data-note);
    position: absolute;
    left: 50%;
    top: 50%;
    transform: translate(-50%, -50%);
    /* Le projet définit ici la pastille orange. */
}
```

Le CSS de séance donne à la pastille orange un fond `#ff5722`, un cercle de 24 px, un texte blanc et une ombre. L'attribut `data-note` alimente le contenu du pseudo-élément. Pour centrer correctement la pastille dans la case, le parent `.fret-cell` doit être positionné (`position: relative`), point à vérifier dans le CSS complet du projet.

Une autre petite amélioration a été identifiée : calculer `midiNote` **une seule fois**, puis le réutiliser pour `getFullNoteName()` et pour la gamme. Le JSX final fourni applique cette simplification.

07 Distinguer les toniques

Afficher uniquement les sept notes de la gamme ne suffit pas toujours pour improviser : les Mi doivent ressortir comme points de résolution. L'intervalle tonique est 0, et toute note Mi a la même classe de hauteur MIDI.

Calcul de la tonique dans le rendu des cases

```
const tonicMidi = 64; // E4
const isInScale = isNoteInNaturalMinor(midiNote, tonicMidi);
const isTonic = midiNote % 12 === tonicMidi % 12;
```

Le test compare les classes de hauteur : `40 % 12` et `64 % 12` valent tous deux 4. La tonique est donc reconnue quelle que soit son octave. Dans le modèle actuel, `isTonic === true` implique `isInScale === true` car l'intervalle zéro est dans la gamme.

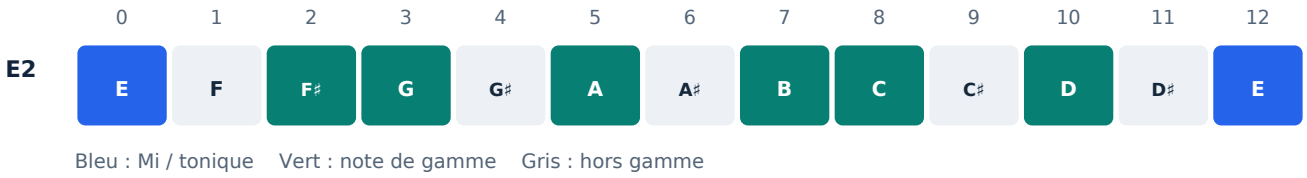
className final fourni (mis sur trois lignes pour la lecture)

```
className={`fret-cell ${isSelected ? 'is-selected' : ''}
  ${isInScale ? 'in-scale' : ''}
  ${isTonic ? 'is-tonic' : ''}`}
```

CSS final de gamme et tonique

```
.in-scale {
    background-color: rgba(3, 76, 52, 0.524);
}
.is-tonic {
    background-color: #2563eb;
}
```

Les deux sélecteurs `.in-scale` et `.is-tonic` ont la même spécificité. La règle `.is-tonic` est placée après : le bleu gagne lorsqu'une case cumule les deux classes. La pastille orange de sélection reste sur `::after`.



> **Choix simple, temporaire.** La tonique MIDI 64 reste codée en dur pendant la séance. La sortir de la boucle sur les cases est une amélioration proposée, non vérifiée ici. La séance 3 rendra la tonique et le type de gamme configurables.

08 Vérifications et difficultés rencontrées

Vérifications manuelles réalisées

Corde 6, case	Note	Couleur / comportement attendu
0	E2	Bleu, tonique ; sélection orange possible
1	F2	Neutre ; sélection orange possible
2	F♯2	Vert ; sélection orange possible
3	G2	Vert ; sélection orange possible
4	G♯2	Neutre ; sélection orange possible
7	B2	Vert
12	E3	Bleu, tonique

Les quatre états d'affichage ont été explicitement testés et confirmés : case neutre, case verte, hors-gamme sélectionnée, dans-gamme sélectionnée. La distinction des toniques a ensuite été intégrée dans le JSX et le CSS, et la séance a été clôturée après confirmation que le build passait.

Commande exécutée en fin de séance — résultat communiqué : OK

```
pnpm build
```

Difficultés pédagogiques réellement rencontrées

Difficulté	Déclic / résolution
Pourquoi ne pas représenter la gamme par les noms de notes ?	Choix des **intervalles depuis la tonique** : invariant musical et base de la transposition.
Différence entre modulo et reste JS	`-21 % 12` renvoie −9 en JS ; normalisation par un double reste pour retrouver 3.
Distance numérique vs résultat de fonction	L'intervalle ramené vaut 10 pour D3 par rapport à E4 ; `includes(10)` renvoie un **booléen**.
État React supplémentaire ?	Non : l'appartenance et la tonique sont des **valeurs dérivées** des données déjà disponibles.
Conflit visuel entre classes CSS ?	Des propriétés/couches séparées et une cascade contrôlée : vert, puis bleu, puis pastille orange.

Ce que le build ne prouve pas

La commande `pnpm build` valide la compilation et les vérifications intégrées au script du projet, mais ne remplace pas des tests unitaires de l'algorithme ni des tests d'interface. Aucune suite de tests automatisés n'a été montrée pendant cette séance : ce travail reste facultatif pour un prochain incrément.

09 Journal des décisions et limites

Décision retenue	Motif	Portée
Offsets en demi-tons depuis la tonique	Calcul direct et transposable	Musique
Nouveau module `scales.ts`	Séparation de la logique métier et de la vue	Organisation
Fonction pure avec deux nombres	Facilité de raisonnement et de test	TypeScript
Modulo positif normalisé	Compatibilité octaves basses/hautes	JavaScript
Un seul état : position choisie	Éviter la duplication des valeurs dérivées	React
Classes CSS cumulables	Préserver une sélection hors gamme	Interface
Bleu pour la tonique	Retrouver les points de résolution	Musique / UI
Tonique fixe = MIDI 64	Limiter volontairement le périmètre	Produit

État technique à la sortie de la séance

- Fonctionnalité acquise : affichage de Mi mineur naturel sur six cordes, 0–12 cases ; affichage de la sélection préservé.
- Architecture conservée : `notes.ts` pour la hauteur et le nom de note, `scales.ts` pour l'appartenance à la gamme, `GuitarNeck.tsx` pour le rendu.
- Pas d'audio, de tablature, de persistance, de backend ou de système abstrait de modes ajouté.
- Prochaine extension proposée : sélectionner la tonique, puis choisir parmi mineur naturel, majeur, dorien et phrygien.

10 Procédure de reconstruction (à partir du chapitre 1)

Cette procédure décrit les changements **exacts à reproduire au niveau des responsabilités**. L'arborescence complète du dépôt et le code intégral de `GuitarNeck.tsx` n'ont pas été fournis dans la séance : les chemins de l'annexe sont donc un exemple cohérent à adapter à l'organisation réelle.

Pré-requis fournis par le chapitre 1

- Un projet React/Vite/pnpm avec TypeScript strict et `noUncheckedIndexedAccess`.
- Le modèle `GuitarString` : `pos: 1 | 2 | 3 | 4 | 5 | 6`, `openMidi: number`.
- Un manche généré avec `guitarStrings.map()` et `fretNumbers.map()`.
- L'état `selectedPosition`, la sélection d'une case et les fonctions de `notes.ts`.

Rejouer l’incrément 2

- **Créer** `src/music/scales.ts` avec `NATURAL_MINOR_INTERVALS`, `modulo` et `isNoteInNaturalMinor` (section 04).
- **Importer** la nouvelle fonction dans le composant de manche ; utiliser la valeur MIDI d’une position pour afficher d’abord « Dans la gamme » ou « Hors gamme » (section 05).
- **Dans la boucle des cases**, calculer une seule fois `midiNote`, puis `isInScale` et ajouter la classe `in-scale` (section 06).
- **Ajouter** `isTonic` par classes de hauteur, puis la classe `is-tonic` (section 07).
- **Déclarer** dans le CSS `.in-scale` avant `.is-tonic`, conserver le pseudo-élément orange et `position: relative` sur la case.
- **Vérifier** les positions de la section 08 et exécuter `pnpm build`.

Annexe A Repartir d’un Vite vierge : exemple autonome

Cette annexe complète la procédure ci-dessus pour un lecteur qui ne dispose pas encore du chapitre 1. Elle fournit une **base minimale reproductible** compatible avec React + TypeScript strict. Elle ne prétend pas refléter ligne pour ligne les fichiers personnels non partagés.

A.1 Initialisation proposée

Commandes reproductibles proposées (non attestées comme commandes exactes de séance 1)

```
pnpm create vite modal-guitar-lab --template react-ts
cd modal-guitar-lab
pnpm install
pnpm dev
```

Dans le tsconfig utilisé par l’application (souvent `tsconfig.app.json` selon la version du template Vite), vérifier que `strict` et `noUncheckedIndexedAccess` sont activés.

Arborescence de démonstration (organisation proposée)

```
src/
  App.tsx
  components/
    GuitarNeck.tsx
    GuitarNeck.css
  music/
    notes.ts
    scales.ts
```

A.2 Modèle et notes — src/music/notes.ts

Première partie : accordage standard

```
export type GuitarString = {
  pos: 1 | 2 | 3 | 4 | 5 | 6;
  openMidi: number;
};

export const guitarStrings: GuitarString[] = [
  { pos: 1, openMidi: 64 }, // E4
  { pos: 2, openMidi: 59 }, // B3
  { pos: 3, openMidi: 55 }, // G3
  { pos: 4, openMidi: 50 }, // D3
  { pos: 5, openMidi: 45 }, // A2
  { pos: 6, openMidi: 40 }, // E2
];
```

Seconde partie : implémentation minimale de reconstruction

```
const NOTE_NAMES = [
  'C', 'C♯', 'D', 'D♯', 'E', 'F',
  'F♯', 'G', 'G♯', 'A', 'A♯', 'B'
] as const;

export function getMidiNote(openMidi: number, fret: number): number {
  return openMidi + fret;
}

export function getNoteName(midi: number): string {
  return NOTE_NAMES[midi % 12] ?? '?';
}
```

```
export function getOctave(midi: number): number {
  return Math.floor(midi / 12) - 1;
}

export function getFullNoteName(midi: number): string {
  return `${getNoteName(midi)}${getOctave(midi)}`;
}
```

Le projet réel possède également `checkMidiRange()`, déjà acquis en séance 1 ; l'exemple simplifié de cette annexe ne le reproduit pas. Les dièses sont notés par ♯ ; les équivalents enharmoniques (Ré♭, etc.) ne sont pas gérés dans cette version.

A.3 Gamme — src/music/scales.ts

Copier le code de la section 04 sans le changer. Il ne dépend ni de React ni d'un accordage donné.

A.4 Manche interactif — src/components/GuitarNeck.tsx

Dans l'exemple ci-dessous, le composant reproduit les fonctionnalités importantes des séances 1 et 2 : sélection d'une position, nom de note, gamme verte, tonique bleue et pastille orange. Les noms d'état et la structure simplifiée peuvent différer du composant réel.

Manche — partie 1 : état et valeurs dérivées

```
import { useState } from 'react';
import { guitarStrings, getMidiNote, getFullNoteName } from '../music/notes';
import type { GuitarString } from '../music/notes';
import { isNoteInNaturalMinor } from '../music/scales';
import './GuitarNeck.css';

type SelectedPosition = {
  string: GuitarString['pos'];
  fret: number;
} | null;

const fretNumbers = Array.from({ length: 13 }, (_, fret) => fret);
```

```
export default function GuitarNeck() {
  const [selectedPosition, setSelectedPosition] =
    useState<SelectedPosition>(null);
  const tonicMidi = 64; // Mi

  const selectedString = selectedPosition
    ? guitarStrings.find(s => s.pos === selectedPosition.string)
    : undefined;
  const selectedMidi = selectedString && selectedPosition
    ? getMidiNote(selectedString.openMidi, selectedPosition.fret)
    : undefined;
  const selectedStatus = selectedMidi === undefined
    ? ''
    : isNoteInNaturalMinor(selectedMidi, tonicMidi)
      ? 'Dans la gamme'
      : 'Hors gamme';
```

Manche — partie 2 : rendu interactif des six cordes

```
  return (
    <section>
      <div className="guitar-neck">
        {guitarStrings.map(string => (
          <div key={string.pos} className="string-row">
            <div className="string-name">
              {getFullNoteName(string.openMidi)}
            </div>
            {fretNumbers.map(fret => {
              const midiNote = getMidiNote(string.openMidi, fret);
              const isSelected = selectedPosition?.string === string.pos
                && selectedPosition.fret === fret;
              const isInScale = isNoteInNaturalMinor(midiNote, tonicMidi);
              const isTonic = midiNote % 12 === tonicMidi % 12;
              const note = isSelected ? getFullNoteName(midiNote) : '';
```

```
              return (
                <div key={fret}
                  className={`fret-cell ${isSelected ? 'is-selected' : ''}
                    ${isInScale ? 'in-scale' : ''}
                    ${isTonic ? 'is-tonic' : ''}`}
                  onClick={() => setSelectedPosition({
                    string: string.pos, fret
                  })}
                  data-note={note}>
                  <div className="string-line" />
                </div>
              );
            })}
```

```
          </div>
        ))}
      </div>
      {selectedMidi !== undefined && (
        <p>NOTE: {getFullNoteName(selectedMidi)} MIDI: {selectedMidi}
          {' — '}{selectedStatus}</p>
      )}
    </section>
  );
}
```

A.5 Styles — src/components/GuitarNeck.css

Styles de grille et de gamme — version autonome

```
.guitar-neck { overflow-x: auto; min-width: 660px; }
.string-row {
  display: grid;
  grid-template-columns: 50px repeat(13, minmax(0, 1fr));
}
.string-name {
  display: flex; align-items: center;
  font: 12px sans-serif; padding-left: 8px;
}
.fret-cell {
  position: relative; height: 48px; cursor: pointer;
  border-right: 2px solid #7b8790;
  border-bottom: 1px solid #dae3e9;
```

```
  background: #e8edf1;
}
.string-line {
  position: absolute; top: 50%; left: 0; right: 0;
  border-top: 1px solid #6b7882;
  pointer-events: none;
}
.in-scale { background-color: rgba(3, 76, 52, 0.524); }
.is-tonic { background-color: #2563eb; }
```

Pastille sélectionnée — variation pédagogique de 30 px

```
.is-selected::after {
  content: attr(data-note);
  position: absolute; left: 50%; top: 50%;
  transform: translate(-50%, -50%);
  width: 30px; height: 30px;
  border-radius: 50%;
  display: flex; align-items: center; justify-content: center;
  background: #ff5722; color: white;
  font: bold 10px sans-serif;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.4);
  z-index: 1;
}
```

Le cercle est ici porté à 30 px pour mieux contenir le nom des notes. Dans le CSS fourni pendant la séance, il mesure 24 px. Les deux choix sont valides tant que l'affichage est lisible.

A.6 Point d'entrée — src/App.tsx

Point d'entrée minimal

```
import GuitarNeck from './components/GuitarNeck';

export default function App() {
  return <main><h1>Modal Guitar Lab</h1><GuitarNeck /></main>;
}
```

Le fichier `main.tsx` du template Vite peut rester inchangé. En cas de styles globaux provenant du template, ajuster ou retirer ceux qui limiteraient la largeur de l'application, sans modifier la logique métier.

Vérification de cette reconstruction pédagogique

```
pnpm build
pnpm dev
```

11 Passage à la séance 3

Le prochain chat vise une seule évolution fonctionnelle à la fois : rendre la **tonique sélectionnable** puis rendre le **type de gamme/mode sélectionnable**. L'objectif est d'éviter de maintenir une fonction distincte pour chaque tonique. Les gammes initialement envisagées sont mineur naturel, majeur, dorien et phrygien.

Points à conserver au prochain chapitre

- Préserver `selectedPosition` : la note sélectionnée et la gamme affichée sont deux notions différentes.
- Réutiliser le modèle par intervalles : les notes de gamme dépendent d'une tonique et d'une suite d'offsets.
- Ne pas ajouter d'audio, de tablature, de backend ou de nouvelle architecture tant que la sélection de gamme n'en a pas besoin.

Fiche de continuité du carnet

Champ	Valeur à reporter
Chapitre	02 — Visualisation des gammes
État	Validé : tests manuels + `pnpm build` OK
Fichiers clés	`notes.ts`, `scales.ts`, `GuitarNeck.tsx`, CSS du manche
Nouvelle API	`isNoteInNaturalMinor(midiNote, tonicMidi): boolean`
Décision ouverte	Tonique actuellement fixée à 64 ; représentation des gammes à généraliser en séance 3

Source et traçabilité. Ce chapitre est basé sur le document de démarrage de Modal Guitar Lab et sur les extraits et validations explicitement échangés pendant la séance 2. Les fichiers complets du dépôt n'ayant pas été fournis, l'annexe A est une reconstruction pédagogique distincte.

Fin du chapitre 02. Le carnet peut accueillir le chapitre 03 sans réécrire celui-ci : toute décision ultérieure modifiant le code doit être documentée dans le nouveau chapitre.